



Gowin 云源软件 Tcl 命令 用户指南

SUG1220-2.0, 2026-06-17

版权所有 © 2026 广东高云半导体科技股份有限公司

GOWIN高云、、Gowin、GOWIN、gowin、GowinSynthesis、高云、云源、小蜜蜂以及晨熙均为广东高云半导体科技股份有限公司注册商标，本手册中提到的其他任何商标，其所有权利属其拥有者所有。未经本公司书面许可，任何单位和个人都不得擅自摘抄、复制、翻译本文档内容的部分或全部，并不得以任何形式传播。

免责声明

本文档并未授予任何知识产权的许可，并未以明示或暗示，或以禁止反言或其它方式授予任何知识产权许可。除高云半导体在其产品的销售条款和条件中声明的责任之外，高云半导体概不承担任何法律或非法律责任。高云半导体对高云半导体产品的销售和 / 或使用不作任何明示或暗示的担保，包括对产品的特定用途适用性、适销性或对任何专利权、版权或其它知识产权的侵权责任等，均不作担保。高云半导体对文档中包含的文字、图片及其它内容的准确性和完整性不承担任何法律或非法律责任，高云半导体保留修改文档中任何内容的权利，恕不另行通知。高云半导体不承诺对这些文档进行适时的更新。

版本信息

日期	版本	说明
2025/08/29	1.0	初始版本。
2025/12/31	1.1	● 更新章节3.1命令分类； ● set_option新增BitStream属性配置选项-bsram_init_print_mode； ● set_option新增Place & Route属性配置选项-co-place_io_registers。
2026/06/17	2.0	● 更新第2章简介； ● 更新第3章章节名称以及内容层级结构； ● 新增第4章Tcl生成IP流程； ● 新增设计文件加密命令encrypt； ● set_option新增综合属性配置选项-opt_goal、-netlist_hierarchy、-dsp_style、-ram_style、-rom_style、-shift_reg、-map_option。

目录

目录	i
图目录	iii
表目录	iv
1 关于本手册	1
1.1 手册内容	1
1.2 相关文档	1
1.3 术语、缩略语	1
1.4 技术支持与反馈	1
2 简介	2
2.1 云源 Tcl 概述	2
2.2 云源 Tcl 运行	2
2.2.1 命令行模式	2
2.2.2 图形界面模式	3
3 Tcl 命令	5
3.1 命令分类	5
3.2 命令列表	6
3.3 命令介绍	7
3.3.1 add_file	7
3.3.2 encrypt	8
3.3.3 create_ipc	9
3.3.4 create_project	10
3.3.5 generate_target	11
3.3.6 get_ips	11
3.3.7 import_files	12
3.3.8 list_property	13
3.3.9 open_project	14
3.3.10 read_ipc	15
3.3.11 report_property	15

3.3.12rm_file	16
3.3.13run	16
3.3.14run close	17
3.3.15saveto	17
3.3.16set_device	18
3.3.17set_file_enable	19
3.3.18set_file_prop	19
3.3.19set_csr	20
3.3.20set_option	21
3.3.21set_property	69
3.3.22source	70
3.3.23write_ip_tcl	70
4 Tcl 生成 IP 流程.....	72

图目录

图 2-1 Tcl 命令行模式	2
图 2-2 Tcl 命令行执行	2
图 2-3 source 命令执行 Tcl 脚本	3
图 2-4 执行程序执行 Tcl 脚本	3
图 2-5 界面 Tcl 控制台	3
图 2-6 界面 Tcl 控制台执行单条命令	3
图 2-7 界面 Tcl 控制台指定 Tcl 脚本	4
图 3-1 Id 显示	5

表目录

表 1-1 术语、缩略语	1
表 3-1 支持 IP 及 Id	6

1 关于本手册

1.1 手册内容

本手册主要描述高云半导体云源软件（以下简称云源）的命令行模式及相关 Tcl 命令的使用方法。

1.2 相关文档

通过登录高云半导体网站 www.gowinsemi.com.cn 可下载、查看以下相关文档：[SUG100](#), [Gowin 云源软件用户指南](#)。

1.3 术语、缩略语

本手册中的相关术语、缩略语及相关释义如表 1-1 所示。

表 1-1 术语、缩略语

术语、缩略语	全称	含义
CRC	Cyclic Redundancy Check	循环冗余校验
FPGA	Field Programmable Gate Array	现场可编程门阵列
GowinSynthesis	GowinSynthesis	高云半导体综合工具
GPA	Gowin Power Analyzer	功耗分析工具
IP Core	Intellectual Property Core	知识产权核
PnR	Place & Route	布局布线
SEU Handler	Single-Event Upsets Handler	单粒子翻转处理器
Tcl	Tool Command Language	工具命令语言

1.4 技术支持与反馈

高云半导体提供全方位技术支持，在使用过程中如有任何疑问或建议，可直接与公司联系：

网址：www.gowinsemi.com.cn

E-mail：support@gowinsemi.com

Tel: +86 755 8262 0391

2 简介

2.1 云源 Tcl 概述

工具命令语言（Tool Command Language，Tcl）是集成在云源中的脚本语言，可以通过 Tcl 命令对云源进行交互以及管理项目。

2.2 云源 Tcl 运行

云源支持在命令行模式下运行 Tcl，也支持在 IDE 下方 Tcl 控制台运行 Tcl。

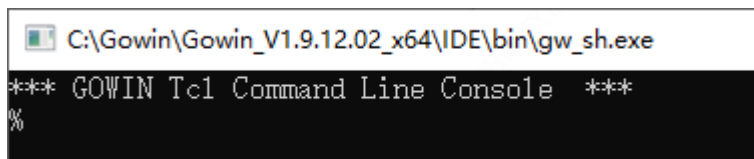
注！

Windows 系统下文件路径的分隔符支持两种书写形式：/或\\，若路径中分隔符为\\，必须添加转义字符\\。

2.2.1 命令行模式

以 Windows 为例，启动安装目录下\\x.x\\IDE\\bin\\gw_sh.exe 进入命令行模式，如图 2-1 所示。

图 2-1 Tcl 命令行模式

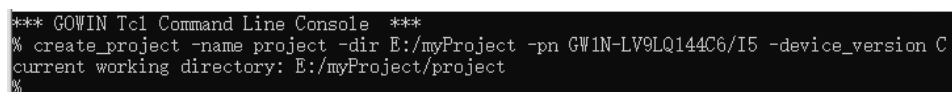


命令行模式支持输入单条 Tcl 命令以及运行 Tcl 脚本两种方式。

输入单条 Tcl 命令

输入 Tcl 命令后通过回车键来执行相关命令，如图 2-2 所示。

图 2-2 Tcl 命令行执行



运行 Tcl 脚本文件

方式一：在命令行模式下通过 **source** 命令指定 Tcl 脚本文件，通过回车执行脚本，如图 2-3 所示。

图 2-3 source 命令执行 Tcl 脚本

```
C:\Gowin\Gowin_V1.9.12.02_x64\IDE\bin>gw_sh.exe
*** GOWIN Tcl Command Line Console ***
% source E:\project.tcl
```

source 命令的具体使用方法参考 [3.3.22 source](#)。

方式二：使用执行程序 **gw_sh.exe** 指定[script file]脚本文件，通过回车执行，如图 2-4 所示。

图 2-4 执行程序执行 Tcl 脚本

```
C:\Gowin\Gowin_V1.9.12.02_x64\IDE\bin>gw_sh.exe project.tcl
```

2.2.2 图形界面模式

以 Windows 为例，启动云源软件，在云源 IDE 下方 Tcl 控制台运行 Tcl，如图 2-5 所示。

图 2-5 界面 Tcl 控制台

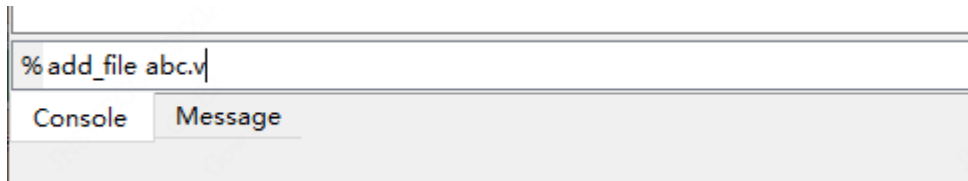


界面 Tcl 控制台同样支持输入单条 Tcl 命令以及运行 Tcl 脚本两种方式。

输入单条 Tcl 命令

该模式下执行方式与命令行模式执行方式一致，如图 2-6 所示。

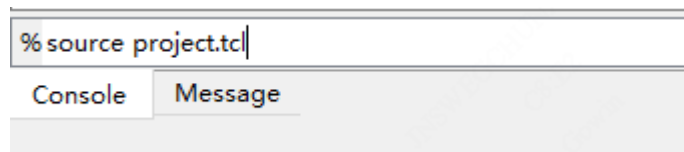
图 2-6 界面 Tcl 控制台执行单条命令



运行 Tcl 脚本文件

与命令行模式下一样通过 **source** 命令指定 Tcl 脚本文件，如图 2-7 所示。

图 2-7 界面 Tcl 控制台指定 Tcl 脚本



3 Tcl 命令

云源支持命令行模式运行。在以下说明中，尖括号<>包含的内容为必须指定的内容，方括号[]包含的内容为可选的内容。创建文件名称时不支持“/”、“*”、“-”、空格等特殊字符，同时指定 IP module_name 时也不支持数字开头。

3.1 命令分类

命令包括用于 IPCore 操作的 IPFlow 分类和工程操作的 Project 分类，支持 IPFlow 分类的 IP 通过 IPCore 界面上的 Id 获取，如图 3-1 Id 显示所示。

图 3-1 Id 显示

Name	Version	Id
> Hard Module		
▼ Soft IP Core		
> AI		
> BackGround Configuration		
> DSP and Mathematics		
> Interface and Interconnect		
▼ Memory Control		
> DDRx SDRAM Memory Interface		
▼ FIFO		
AXI-Stream FIFO	1.0	
FIFO	1.1	fifo
FIFO HS	1.0	fifo_hs
FIFO SC	1.1	fifo_sc
FIFO SC HS	1.1	fifo_sc_hs

已支持的 IP 如表 3-1 所示。

表 3-1 支持 IP 及 Id

IP 名称	Id
PLL_ADV	clock_plladv
MIPI_DPHY_RX	mipi_dphy_rx
MIPI_DPHY	mipi_dphy
MIPI_CPHY	mipi_cphy
ROM	rom
MIPI RX Advance	mipi_rx_advance
MIPI TX Advance	mipi_tx_advance
DDR3 Memory Interface	ddr3
FIFO	fifo
FIFO SC	fifo_sc
FIFO HS	fifo_hs
FIFO SC HS	fifo_sc_hs
DP	dp
SDP	sdp
SDP36KE	sdp36ke
SP	sp
Revised DP	revised_dp
Revised SP	revised_sp

[IPFlow](#)[Project](#)

3.2 命令列表

IPFlow:

[create_ipc](#)[generate_target](#)[get_ips](#)[list_property](#)[read_ipc](#)[report_property](#)[set_property](#)[source](#)[write_ip_tcl](#)

Project:

[add_file](#)

[create_project](#)
[import_files](#)
[open_project](#)
[rm_file](#)
[run](#)
[run close](#)
[saveto](#)
[set_device](#)
[set_file_enable](#)
[set_file_prop](#)
[set_option](#)
[source](#)

3.3 命令介绍

3.3.1 add_file

指定要添加到工程的设计文件。

语法

add_file [-type] [-disable] [-h/--help] <file>

使用

名称	描述
[-type]	添加的设计文件类型
[-disable]	设置为失效状态
[-h/--help]	显示帮助信息
<file>	要添加的文件

分类

[Project](#)

描述

添加设计文件，支持相对路径和绝对路径，相对路径形式在云源界面下相对的是当前工程的路径，在命令行模式下相对的是启动 gw_sh 时的路径。

参数

- <file>要添加的设计文件，可以指定多个，多个文件以空格分隔；
- [-type <type>] add_file 命令会根据文件扩展名自动判断文件类型，也可以使用该选项来明确指定文件类型，支持的文件类型有 verilog、vhdl、sv、vg、cst、sdc、gao、gpa、gsc 等；

- **[-disable]**将添加的文件置为失效状态。处于失效状态的文件仅添加到文件列表中，不参与流程实现。相关命令：**set_file_enable**;
- **[-h/--help]**显示帮助信息。

示例

```
add_file abc.v
add_file -type vhd1 1.vhd 2.vhd 3.vhd
add_file D:/gowin_project/top.v
add_file D:\\gowin_project\\top.v
```

3.3.2 encrypt

对工程管理区内的 HDL 设计文件进行加密。

语法

```
encrypt [-verilog | -vhd1 ] <-i inputFile>[-o outputFile] [-l/list fileList] [-h/--help]
```

使用

名称	描述
[-verilog -vhd1]	指定要加密的文件类型
<-i inputFile>	指定要加密的文件
[-o outputFile]	指定加密后的输出文件
[-l/list fileList]	指定要加密的文件列表
[-h/--help]	显示帮助信息

分类

[Project](#)

描述

对工程管理区内的单个或多个 HDL 设计文件进行加密，并输出加密后的文件。该加密功能是对 HDL 设计文件内容的全加密，且不会改变设计文件的 **module** 层级架构及内容。

参数

- **[-verilog | -vhd1]** 要加密的文件类型，可以不指定，会根据文件扩展名自动判断文件类型；
- **<-i inputFile>** 要加密的输入文件，支持相对路径和绝对路径，使用相对路径时相对的是工程文件所在路径；
- **[-o outputFile]** 加密后的输出文件。若不指定该项，默认放在输入文件所在路径下；
- **[-l/list fileList]** 要加密的文件列表，可同时对文件列表中的多个文件进行加密；

- [-h/--help]显示帮助信息。

示例

```
encrypt -verilog -i D:/gowin_project/top.v -o D:/gowin_project/top.vp
```

3.3.3 create_ipc

生成默认配置的 IPC 文件。

语法

```
create_ipc -name <ipName> -module_name <moduleName> [-language <arg>] [-file_name <fileName>] [-dir <path>] [-force]
```

使用

名称	描述
-name	IP name
-module_name	将要创建的IP的module名称
[-language]	IP文件、模板文件以及仿真文件的语言
[-file_name]	IP文件的名称
[-dir]	IP生成路径
[-force]	覆盖已有文件

分类

[IPFlow](#)

描述

该命令为 IPCoreGenerator 中的 IP 创建 IPC 文件。

参数

- -name <ipName>指定 IP 的名称，该名称可通过 IP Core Generator 界面的 Id 获取；
- -module_name <moduleName>指定要创建的 IP 的 module 名称；
- [-language <arg>]指定生成的 IP 文件、模板文件、仿真文件的语言（Verilog/VHDL），若不指定则默认使用 Verilog；
- [-file_name <fileName>]指定生成的 IP 文件的名称，若不指定则与 module_name 指定的名称一致；
- [-dir] <path>指定 IP 生成路径，如果没有指定，生成到当前工程的 src 文件夹下；
- [-force]覆盖已有的文件。

示例

下面的示例在当前工程的 src 文件夹下创建由-name 指定的 IP，并指定模块名称、语言以及文件名称：

```
create_ipc -name fifo -module_name FIFO_Top -language Verilog -
```

file_name fifo

另请参阅

[generate_target](#)

3.3.4 create_project

创建工程。

语法

```
create_project [-name <prjName>] [-dir <path>] [-pn <pnName>] [-device_version <arg>] [-force] [-h/--help]
```

使用

名称	描述
[-name]	要创建工程的名称
[-dir]	要创建的工程路径
[-pn]	要创建工程的Part Number
[-device_version]	要创建工程的device version
[-force]	覆盖已有文件
[-h/--help]	显示帮助信息

分类

[Project](#)

描述

新建工程，文件路径形式请参考 [add_file](#)。

参数

- [-name <prjName>]指定要创建工程的名称；
- [-dir <path>]指定创建的工程所在的路径，如果指定的路径不存在则创建一个新路径。如果没有指定该选项，则默认使用当前工作路径或 tcl 脚本所在的路径；
- [-pn <pnName>]指定创建工程的 Part Number；
- [-device_version <arg>]指定 Part Number 对应的 device version，对于只有初始版本的器件，device version 的值为 NA；
- [-force]该选项会覆盖已有的工程；
- [-h/--help]显示帮助信息。

示例

```
create_project -name prj0 -dir D:/tclprj -pn GW1N-UV4LQ144C6/I5 -device_version B
```

```
create_project -name prjlest -pn GW1N-UV4LQ144C6/I5 -device_version B -force
```

3.3.5 generate_target

为指定的 IP 生成 IP 设计文件、模板文件以及仿真文件。

语法

```
generate_target <objects> [-force]
```

使用

名称	描述
<objects>	指定要生成IP文件的IP
[-force]	覆盖已有文件

分类

[IPFlow](#)

描述

该命令为指定的 IP（`get_ips`）生成 IP 设计文件、模板文件以及仿真文件，并将 IP 设计文件添加到当前工程中。

参数

- <objects>用于指定生成设计文件的一个或多个 IP，一个 IP 通过 `[get_ips module_name]`指定，如果是指定多个 IP，则使用`[get_ips module_name0 module_name1 ...]`。
- [-force]覆盖已有文件

示例

下面的示例为指定的 IP 生成相关文件并将设计文件加载到当前工程中：

```
generate_target [get_ips FIFO_Top]
```

另请参阅

- [generate_target](#)
- [create_ipc](#)
- [read_ipc](#)

3.3.6 get_ips

指定 IP。

语法

```
get_ips <module_name>
```

使用

名称	描述
<module_name>	指定 IP 的 module_name

分类

[IPFlow](#)

描述

指定当前工程中的 IP。

参数

<module_name>指定 IP 的 module_name，可以是一个也可以是多个。

示例

下面的示例为指定当前工程的一个 IP：

```
get_ips FIFO_Top
```

下面的示例为指定当前工程的多个 IP：

```
get_ips FIFO_Top FIFO_Top_1
```

另请参阅

- [generate_target](#)
- [list_property](#)
- [report_property](#)
- [set_property](#)

3.3.7 import_files

拷贝文件或目录到当前工程。

语法

```
import_files [-file <file>] [-dir <path>] [-fileList <fileList>] [-force] [-h/--help]
```

使用

名称	描述
[-file]	指定要拷贝的文件
[-dir]	指定要拷贝的路径
[-fileList]	指定列表文件
[-force]	覆盖已有同名文件
[-h/--help]	显示帮助信息

分类

[Project](#)

描述

将文件或目录拷贝到当前工程路径/src 下。 -file、-dir 和-fileList 后面的 value 可以跟绝对路径，也可以是相对路径。如果是通过 gw_sh.exe 执行 tcl 脚本，则相对的是 tcl 脚本的路径，如果是通过云源软件中的 console 窗口执行 import files 命令，则相对的是当前工作目录(pwd)。如果 impot_files 后面没有跟任何选项，则默认将 add_file 命令指定的所有文件拷贝到工程路径/src 下。

参数

- [-file <file>]将一个或多个文件添加到工程路径/src 下；
- [-dir <path>]将路径下的所有文件及子文件夹添加到工程路径/src 下；
- [-fileList <fileList>]指定列表文件，文件中每一行为一个要添加的工程文件，内容示例如下：
 - D:/test1.v
 - D:/test2.v
 - 该选项可将文件中每一行所指定的文件都添加到工程路径/src 下；
- [-force]覆盖工程路径/src 下的同名文件；
- [-h/--help] 显示帮助信息。

示例

```
import_files -file D/test1 .v -force
import_files -file D:/test1 .v D:/test2.v -force
import_files -dir D:/sourceFile
import_files -fileList log, log 文件的内容为:
D:/Test1.v
D:/Test2.v
```

3.3.8 list_property

列举 IP 的配置选项。

语法

```
list_property <object>
```

使用

名称	描述
<object>	指定查询的 IP

分类

[IPFlow](#)

描述

获取指定 IP 的所有选项的列表。

参数

<object>指定查询选项的某个 IP，通过[get_ips module_name]指定。

示例

下面的示例列出指定 IP 的配置选项：

```
list_property [get_ips FIFO_Top]
```

另请参阅

- [report_property](#)
- [set_property](#)

3.3.9 open_project

打开工程。

语法

```
open_project <file>] [-pn] [-device_version] [-h/--help]
```

使用

名称	描述
<file>	指定工程文件
[-pn]	指定Part Number信息
[-device_version]	指定工程的device version
[-h/--help]	显示帮助信息

分类

[Project](#)

描述

打开工程，可以对要打开的工程指定一个新的 Part Number。文件路径形式请参考 [add_file](#)。

参数

- <file>指定要打开的工程文件名称；
- [-pn] 指定工程的 Part Number 信息；
- [-device_version] 指定工程器件的版本号信息；
- [-h/--help] 显示帮助信息。

示例

```
open_project D:\test.gprj
```

3.3.10 read_ipc

读取 IPC 文件。

语法

```
read_ipc <file>
```

使用

名称	描述
<file>	IPC 文件

分类

[IPFlow](#)

描述

读取指定的 IPC 文件，通过 IPC 文件配置选项，文件路径形式请参考 [add file](#)。

参数

<file>指定要打开的 IPC 文件。

示例

下面的示例为读取指定路径下的 IPC 文件：

```
read_ipc D:/gowin_project/src/fifo/fifio.ipc
```

另请参阅

[generate_target](#)

3.3.11 report_property

获取 IP 的选项名称，选项类型以及选项值。

语法

```
report_property <object>
```

使用

名称	描述
<object>	指定查询选项的 IP

分类

[IPFlow](#)

描述

获得指定 IP 的选项名称，选项类型以及选项值。

参数

<object>指定需要查询选项的某个 IP，通过[get_ips module_name]指定。

示例

下面的示例列出指定 IP 的所有选项以及当前选项值：

```
report_property [get_ips FIFO_Top]
```

另请参阅

- [set_property](#)
- [list_property](#)

3.3.12 rm_file

移除设计文件。

语法

```
rm_file [-h/--help ] <files>
```

使用

名称	描述
<-files>	要移除的设计文件
[-h/--help]	显示帮助信息

分类

[Project](#)

描述

移除设计文件，文件路径形式请参考 [add_file](#)。

参数

- <-files>指定要移除的设计文件，可以指定多个文件，以空格分隔；
- [-h/--help] 显示帮助信息。

示例

```
rm_file a.v
rm_file a.v b.v c.v
rm_file D:/gowin_project/top.v
rm_file D:\\gowin_project\\top.v
```

3.3.13 run

运行流程。

语法

```
run [-h/--help] <syn/pnr/all>
```

使用

名称	描述
<syn/pnr/all>	指定要运行的流程名称

名称	描述
<code>[-h/--help]</code>	显示帮助信息

分类

[Project](#)

描述

运行全部流程或某个流程。

参数

- `<syn/pnr/all>` 指定要运行的流程名称，可以被执行的流程名称有 **syn** 和 **pnr**，分别表示综合及布局布线。也可以指定 **all**，表示运行全部的流程；
- `[-h/--help]` 显示帮助信息。

示例

```
run pnr
run all
```

3.3.14 run close

关闭工程。

语法

```
run close
```

分类

[Project](#)

描述

关闭当前工程。

示例

```
run close
```

3.3.15 saveto

保存工程数据到 Tcl 脚本。

语法

```
saveto [-all_options] [-h/--help] <file>
```

使用

名称	描述
<code>[-all_options]</code>	保存所有的选项信息
<code>[-h/--help]</code>	显示帮助信息
<code><file></code>	保存的文件名

分类

[Project](#)

描述

将当前工程设计数据保存到 Tcl 脚本。文件路径形式请参考 [add file](#)。

参数

- [-all_options] saveto 命令默认只保存修改过的选项信息，即与默认值不同的选项。可以通过指定 -all_options 来保存所有的选项信息；
- [-h/--help] 显示帮助信息；
- <file> 保存的文件名。

示例

```
saveto project.tcl
saveto -all_options project.tcl
saveto -all_options D:/gowin_project/project.tcl
saveto -all_options D:\\gowin_project\\project.tcl
```

3.3.16 set_device

设置芯片型号。

语法

```
set_device [-device_version <value>] [-h/--help ] <part number>
```

使用

名称	描述
[-device_version <value>]	要设置的device version
[-h/--help]	显示帮助信息
<part number>	要设置的Part Number

分类

[Project](#)

描述

设置芯片型号。

参数

- <part number> 指定目标器件的 Part Number，如 GW1N-UV4LQ144C6/I5；
- [-device_version<value>] 指定器件版本号信息，支持的版本号取值有 NA|B|C|D；
- [-h/--help] 显示帮助信息。

示例

```
set_device GW1N-LV1CS30C6/I5
set_device -device_version C GW1N-UV4LQ144C6/I5
```

3.3.17 set_file_enable

设置文件使能属性。

语法

```
set_file_enable <file> <true|false> [-h/--help]
```

使用

名称	描述
<file>	指定要设置的设计文件
<true false>	文件是否被使用
[-h/--help]	显示帮助信息

分类

[Project](#)

描述

设置文件是否可以被使用，文件路径形式请参考 [add_file](#)。

参数

- <file> 指定要设置的文件；
- <true|false> true 表示文件可以被使用，false 表示不可被使用；
- [-h/--help] 显示帮助信息。

示例

```
set_file_enable top.v false
set_file_enable D:/gowin_project/top.v
set_file_enable D:\\gowin_project\\top.v
```

3.3.18 set_file_prop

设置文件属性。

语法

```
set_file_prop <file> [-lib <name>] [-h/--help]
```

使用

名称	描述
<file>	指定要设置的设计文件
[-lib <name>]	设置文件的library name
[-h/--help]	显示帮助信息

分类

[Project](#)

描述

设置文件属性，文件路径形式请参考 [add_file](#)。

参数

- <file> 指定要设置的文件，可以指定多个，多个文件之间以空格分隔；
- [-lib <name>] 设置文件的 library name。该选项仅对 VHDL 类型的文件有效；
- [-h/--help] 显示帮助信息。

示例

```
set_file_prop -lib work top .vhd
set_file_prop -lib work D:/gowin_project/top.vhd
set_file_prop -lib work D:\\gowin_project\\top.vhd
```

3.3.19 set_csr

指定 csr 文件。

语法

```
set_csr [-h/--help ] <file>
```

使用

名称	描述
<file>	要指定的csr文件
[-h/--help]	显示帮助信息

分类

[Project](#)

描述

指定 csr 文件，文件路径形式请参考 [add_file](#)。

参数

- <files>指定要使用的 csr 文件；
- [-h/--help]显示帮助信息。

示例

```
set_csr a.csr
set_csr D:/gowin_project/a.csr
set_csr D:\\gowin_project\\a.csr
```

3.3.20 set_option

设置工程相关的属性配置及流程选项。

语法

```
set_option [options] [-h/--help]
```

使用

名称	描述
[options]	指定配置或流程选项
[-h/--help]	显示帮助信息

分类

[Project](#)

描述

设置工程相关的属性配置及流程选项。

参数

- [options] 指定配置或流程选项；
- [-h/--help] 显示帮助信息。

Global 属性配置

-output_base_name

指定输出文件的文件名。

语法

```
-output_base_name <name>
```

使用

名称	描述
<name>	指定输出文件的文件名

分类

[Project](#)

描述

指定输出文件的文件名。此选项仅指定文件的 **base name**，不同流程会根据输出文件的类型使用合适的扩展名。如 **-output_base_name abc**，则 **gowinsynthesis** 综合产生的网表文件名为 **abc.vg**；

参数

<name>指定输出文件的文件名。

示例

```
set_option -output_base_name abc
```

-global_freq

指定 frequency 值。

语法

-global_freq <default|value>

使用

名称	描述
<default value>	指定frequency值

分类

[Project](#)

描述

指定 frequency 值，默认为 default（小蜜蜂家族芯片默认是 50MHz，晨熙家族芯片默认是 100MHz）。

参数

<default|value>frequency 值。

示例

set_option -global_freq 80

综合属性配置**-synthesis_tool**

指定综合工具。

语法

-synthesis_tool <tool>

使用

名称	描述
<tool>	指定综合工具

分类

[Project](#)

描述

指定综合工具 GowinSynthesis。

参数

<tool>指定综合工具 GowinSynthesis。

示例

set_option -synthesis_tool GowinSynthesis

-opt_goal

指定综合优化目标。

语法

`-opt_goal <auto|area|timing>`

使用

名称	描述
<code><auto area timing ></code>	auto: 自动优化模式; area: 面积优先模式; timing: 时序优先模式。

分类

[Project](#)

描述

指定综合优化目标，默认为 `auto`。

参数

`<auto|area|timing>`综合优化目标选项。

示例

```
set_option -opt_goal timing
```

`-top_module`

指定 Top Module/Entity。

语法

`-top_module <name>`

使用

名称	描述
<code><name></code>	指定top module

分类

[Project](#)

描述

指定 top module。

参数

`<name>`指定 top module。

示例

```
set_option -top_module test
```

`-include_path`

指定包含路径。

语法

`-include_path <path or path list>`

使用

名称	描述
<code><path or path list></code>	指定包含路径

分类

[Project](#)

描述

指定包含路径。当指定多个包含路径时，路径之间需要使用分号进行分隔，并使用一对大括号{}包含所有的路径信息，如`-include_path {/path1;/path2;/path3}`。支持相对路径和绝对路径，相对路径为相对程序当前运行路径。

参数

`<path or path list>`指定包含路径。

示例

```
set_option -include_path D:/project
```

-verilog_std

指定 Verilog 语言标准。

语法

`-verilog_std<v1995|v2001|sysv2017>`

使用

名称	描述
<code><v1995 v2001 sysv2017></code>	指定Verilog语言标准

分类

[Project](#)

描述

指定 Verilog 语言标准：Verilog 95/Verilog 2001/System Verilog 2017，默认为 Verilog 2001。

参数

`<v1995|v2001|sysv2017>`指定 Verilog 语言标准。

示例

```
set_option -verilog_std v1995
```

-vhdl_std

指定 VHDL 语言标准。

语法

```
-vhdl_std <vhdl1993|vhdl2008|vhdl2019>
```

使用

名称	描述
<vhdl1993 vhdl2008 vhdl2019>	指定VHDL语言标准

分类

[Project](#)

描述

指定 VHDL 语言标准：VHDL 1993/VHDL 2008/VHDL 2019，默认为 VHDL1993。

参数

<vhdl1993|vhdl2008|vhdl2019>指定 VHDL 语言标准。

示例

```
set_option -vhdl_std vhdl2008
```

-print_all_synthesis_warning <0|1>

指定是否打印所有的综合 warning 信息，默认为 0。

语法

```
-print_all_synthesis_warning <0|1>
```

使用

名称	描述
<0 1>	0: 不会打印所有的warning信息; 1: 会打印所有的warning信息。

分类

[Project](#)

描述

指定是否打印所有的综合 warning 信息，默认为 0。

参数

<0|1>指定是否打印所有的综合 warning 信息。

示例

```
set_option -print_all_synthesis_warning 1
```

-disable_io_insertion

启用或禁用 I/O 插入的使能控制。

语法

-disable_io_insertion <0|1>

使用

名称	描述
<0 1>	0: 启用I/O插入的使能控制; 1: 禁用I/O插入的使能控制。

分类

[Project](#)

描述

启用或禁用 I/O 插入的使能控制，默认为 0。

参数

<0|1>启用或禁用 I/O 插入的使能控制。

示例

```
set_option -disable_io_insertion 1
```

-netlist_hierarchy

指定生成的网表是采用层次化结构还是扁平化结构。

语法

-netlist_hierarchy <0|1>

使用

名称	描述
<0 1>	0: 网表采用扁平化结构; 1: 网表采用层次化结构。

分类

[Project](#)

描述

指定生成的网表是采用层次化结构还是扁平化结构，默认为 1。

参数

<0|1>指定生成的网表是采用层次化结构还是扁平化结构。

示例

```
set_option -netlist_hierarchy 1
```

-looplimit <value>

RTL 中默认的编译器循环限制值。

语法

-looplimit <value>

使用

名称	描述
<value>	loopleftimit值

分类

[Project](#)

描述

设置 RTL 中默认的编译器循环限制值，默认值为 2000。

参数

<value> RTL 中默认的编译器循环限制值。

示例

```
set_option -loopleftimit 1000
```

-max_fanout <value>

设置扇出值。

语法

```
-max_fanout <value>
```

使用

名称	描述
<value>	max_fanout值

分类

[Project](#)

描述

设置一个输入端口、net 或寄存器输出端的扇出值，默认为 1000。

参数

<value>输入端口、net 或寄存器输出端的扇出值。

示例

```
set_option -max_fanout 2000
```

-rw_check_on_ram

在 RAM 周围插入旁路逻辑。

语法

```
-rw_check_on_ram <0|1>
```

使用

名称	描述
<0 1>	0: 不启用该选项;

名称	描述
	1: 启用该选项。

分类

[Project](#)

描述

如果 RAM 存在读或写冲突，使用该选项后会在 RAM 周围插入旁路逻辑以防止仿真不匹配，默认为 0。

参数

<0|1> 启用或禁用在 RAM 周围插入旁路逻辑使能控制。

示例

```
set_option -rw_check_on_ram 1
```

-dsp_style

指定 DSP 模块的实现方式。

语法

```
-dsp_style <auto|logic|dsp>
```

使用

名称	描述
<auto logic dsp>	auto: 自动模式; logic: 逻辑实现; dsp: 专用DSP模块实现。

分类

[Project](#)

描述

指定 DSP 模块的实现方式，默认为 auto。

参数

<auto|logic|dsp> 指定 DSP 模块的实现方式。

示例

```
set_option -dsp_style logic
```

-ram_style

指定 RAM 模块的实现方式。

语法

```
-ram_style <auto|block_ram|distributed_ram|registers>
```

使用

名称	描述
<auto block_ram distributed_ram registers>	auto: 自动模式; block_ram: 块状BRAM实现; distributed_ram: 分布式RAM实现; registers: 寄存器实现。

分类

[Project](#)

描述

指定 RAM 模块的实现方式，默认为 auto。

参数

<auto|block_ram|distributed_ram|registers>指定 RAM 模块的实现方式。

示例

```
set_option -ram_style registers
```

-rom_style

指定 ROM 模块的实现方式。

语法

```
-rom_style <auto|block_rom|distributed_rom|logic>
```

使用

名称	描述
<auto block_rom distributed_rom logic>	auto: 自动模式; block_rom: 块状存储实现; distributed_rom: 分布式ROM实现; logic: 组合逻辑实现。

分类

[Project](#)

描述

指定 ROM 模块的实现方式，默认为 auto。

参数

<auto|block_rom|distributed_rom|logic>指定 ROM 模块的实现方式。

示例

```
set_option -rom_style logic
```

-shift_reg

指定移位寄存器的实现方式。

语法

-shift_reg <auto|block_ram|distributed_ram|registers>

使用

名称	描述
<auto block_ram distributed_ram registers>	auto: 自动模式; block_ram: 块状BRAM实现; distributed_ram: 分布式RAM实现; registers: 寄存器实现。

分类

[Project](#)

描述

指定移位寄存器的实现方式，默认为 **auto**。

参数

<auto|block_ram|distributed_ram|registers>指定移位寄存器的实现方式。

示例

```
set_option -shift_reg registers
```

-map_option

指定 LUT 映射优化模式。

语法

-map_option <1|2|3|4>

使用

名称	描述
<1 2 3 4>	1: 默认映射模式; 2: 不增加LUT资源占用的LUT5导向映射优化模式; 3: 允许增加LUT资源占用的LUT5导向映射模式以提高时序; 4: 允许增加LUT资源的LUT5/LUT6映射模式以获取最佳时序性能。

分类

[Project](#)

描述

指定 LUT 映射优化模式，默认为 **1**。

参数

<1|2|3|4>指定 LUT 映射优化模式。

示例

```
set_option -map_option 2
```

Place & Route 属性配置

-VCCX

指定 vccx 值

语法

```
-vccx <value>
```

使用

名称	描述
<value>	指定vccx值

分类

[Project](#)

描述

指定 vccx 值。

参数

<value>指定 vccx 值。

示例

```
set_option -vccx 3.3
```

-VCC

语法

```
-vcc <value>
```

使用

名称	描述
<value>	指定vcc值

分类

[Project](#)

描述

指定 vcc 值。

参数

<value>指定 vcc 值。

示例

```
set_option -vcc 3.3
```

-gen_sdf

是否产生 SDF 文件。

语法

```
-gen_sdf <0|1>
```

使用

名称	描述
<0 1>	0: 不产生SDF文件; 1: 产生SDF文件。

分类

[Project](#)

描述

指定 Place & Route 是否产生 SDF 文件，默认为 0。

参数

<0|1>产生 SDF 文件使能控制。

示例

```
set_option -gen_sdf 1
```

-gen_io_cst

是否产生文件名为*.io.cst 的 port 端口的物理约束文件。

语法

```
-gen_io_cst <0|1>
```

使用

名称	描述
<0 1>	0: 不产生*.io.cst文件; 1: 产生*.io.cst文件。

分类

[Project](#)

描述

指定 Place & Route 是否产生文件名为*.io.cst 的 port 端口的物理约束文件，默认为 0。

参数

<0|1>产生 port 端口的物理约束文件使能控制。

示例

```
set_option -gen_io_cst 1
```

-gen_ibis

是否产生文件名为*.ibs 的输入/输出缓冲区信息指定文件。

语法

-gen_ibis <0|1>

使用

名称	描述
<0 1>	0: 不产生*.ibs文件; 1: 产生*.ibs文件。

分类

[Project](#)

描述

指定 Place & Route 是否产生文件名为*.ibs 的输入/输出缓冲区信息指定文件，默认为 0。

参数

<0|1>产生输入/输出缓冲区信息指定文件使能控制。

示例

```
set_option -gen_ibis 1
```

-gen_posp

是否产生器件布局文件。

语法

-gen_posp <0|1>

使用

名称	描述
<0 1>	0: 不产生*.posp文件; 1: 产生*.posp文件。

分类

[Project](#)

描述

指定 Place & Route 是否产生文件名为*.posp 的器件布局文件，文件中只含有 BSRAM 布局信息，默认为 0。

参数

<0|1>产生器件布局文件使能控制。

示例

```
set_option -gen_posp 1
```

-gen_text_timing_rpt

产生文本格式的时序报告。

语法

-gen_text_timing_rpt <0|1>

使用

名称	描述
<0 1>	0: 不产生*.tr文件; 1: 产生*.tr文件。

分类

[Project](#)

描述

指定 Place & Route 是否产生文件名为*.tr 的文本格式的时序报告，默认为 0。

参数

<0|1>产生文本格式的时序报告使能控制。

示例

```
set_option -gen_text_timing_rpt 1
```

-gen_verilog_sim_netlist

是否产生 Verilog 语言的时序仿真模型文件。

语法

-gen_verilog_sim_netlist <0|1>

使用

名称	描述
<0 1>	0: 不产生*.vo文件; 1: 产生*.vo文件。

分类

[Project](#)

描述

指定 Place & Route 是否产生文件名为*.vo 的 Verilog 语言的时序仿真模型文件，默认为 0。

参数

<0|1>产生 Verilog 语言的时序仿真模型文件使能控制。

示例

```
set_option -gen_verilog_sim_netlist 1
```

-gen_vhdl_sim_netlist

是否产生 VHDL 语言的时序仿真模型文件。

语法

-gen_vhdl_sim_netlist <0|1>

使用

名称	描述
<0 1>	0: 不产生*.vho文件; 1: 产生*.vho文件。

分类

[Project](#)

描述

指定 Place & Route 是否产生文件名为*.vho 的 VHDL 语言的时序仿真模型文件，默认为 0。

参数

<0|1>产生 VHDL 语言的时序仿真模型文件使能控制。

示例

```
set_option -gen_vhdl_sim_netlist 1
```

-show_init_in_vo

将默认初始值添加到时序仿真模型文件中的 instance 中。

语法

-show_init_in_vo <0|1>

使用

名称	描述
<0 1>	0: 不会将默认初始值添加到时序仿真模型文件中的instance中; 1: 将默认初始值添加到时序仿真模型文件中的instance中。

分类

[Project](#)

描述

将默认初始值添加到生成的 Place & Route 时序仿真模型文件中的 instance 中，默认为 0。

参数

<0|1>将默认初始值添加到生成的 Place & Route 时序仿真模型文件中的 instance 中的使能控制。

示例

```
set_option -show_init_in_vo 1
```

-show_all_warn

是否输出所有的 warning 信息。

语法

-show_all_warn<0|1>

使用

名称	描述
<0 1>	0: 不会输出Place & Route运行时所有的warning信息; 1: 输出Place & Route运行时所有的warning信息。

分类

[Project](#)

描述

Place & Route 运行时输出所有的 warning 信息，默认为 0。

参数

<0|1>输出 Place & Route 运行时所有的 warning 信息使能控制。

示例

```
set_option -show_all_warn 1
```

-timing_driven

是否对布局布线进行时序驱动优化。

语法

-timing_driven <0|1>

使用

名称	描述
<0 1>	0: 不会进行布局布线时序驱动优化; 1: 进行布局布线时序驱动优化。

分类

[Project](#)

描述

Place & Route 运行时对布局布线进行时序驱动优化，默认为 1。

参数

<0|1>运行布局布线时是否启动时序驱动优化使能控制。

示例

```
set_option -timing_driven 1
```

-cst_warn_to_error

将物理约束警告提升为错误信息。

语法

```
-cst_warn_to_error <0|1>
```

使用

名称	描述
<0 1>	0: 不会将物理约束警告提升为错误信息; 1: 将物理约束警告提升为错误信息。

分类

[Project](#)

描述

将 Place & Route 运行时的物理约束警告提升为错误信息，默认为 1。

参数

<0|1>是否将 Place & Route 运行时的物理约束警告提升为错误信息使能控制。

示例

```
set_option -cst_warn_to_error 1
```

-rpt_auto_place_io_info

报告自动布局的 IO 位置信息。

语法

```
-rpt_auto_place_io_info <0|1>
```

使用

名称	描述
<0 1>	0: 不会报告自动布局的IO位置信息; 1: 报告自动place的IO位置信息。

分类

[Project](#)

描述

Place & Route 运行时报告自动布局的 IO 位置信息，默认为 0。

参数

<0|1>是否报告自动 place 的 IO 位置信息使能控制。

示例

```
set_option -cst_warn_to_error 1
```

-place_option

布局算法选项。

语法

-place_option <0|1|2|3|4>

使用

名称	描述
<0 1 2 3 4>	0: 采用默认布局算法; 1: 采用布局算法 1; 2: 采用布局算法 2。 3: 采用布局算法 3; 4: 采用布局算法 4。

分类

[Project](#)

描述

布局算法选项，默认为 0。

参数

<0|1|2|3|4>布局算法选项。

示例

```
set_option -place_option 1
```

-route_option

布线算法选项。

语法

-route_option <0|1|2>

使用

名称	描述
<0 1 2>	0: 采用默认布线算法; 1: 采用布线算法1; 2: 采用布线算法2。

分类

[Project](#)

描述

布线算法选项，默认为 0。

参数

<0|1|2>布线算法选项。

示例

```
set_option -route_option 1
```

-ireg_in_iob

将输入 Buffer 连接的寄存器布局到 IOB 上。

语法

-ireg_in_job <0|1>

使用

名称	描述
<0 1>	0: 不将输入Buffer连接的寄存器布局到IOB上; 1: 将输入Buffer连接的寄存器布局到IOB上。

分类

[Project](#)

描述

启用该选项，Place & Route 会将输入 Buffer 连接的寄存器布局到 IOB 上，默认为 1。

参数

<0|1>将输入 Buffer 连接的寄存器布局到 IOB 上使能控制。

示例

```
set_option -ireg_in_job 1
```

-oreg_in_job

将输出/三态 Buffer 连接的寄存器布局到 IOB 上。

语法

-oreg_in_job <0|1>

使用

名称	描述
<0 1>	0: 不将输出/三态Buffer连接的寄存器布局到IOB上; 1: 将输出/三态Buffer连接的寄存器布局到IOB上。

分类

[Project](#)

描述

启用该选项，Place & Route 会将输出/三态 Buffer 连接的寄存器布局到 IOB 上，默认为 1。

参数

<0|1>将输出/三态 Buffer 连接的寄存器布局到 IOB 上使能控制。

示例

```
set_option -oreg_in_job 1
```

-ioreg_in_iob

将双向 Buffer 连接的寄存器布局到 IOB 上。

语法

-ioreg_in_iob <0|1>

使用

名称	描述
<0 1>	0: 不将双向Buffer连接的寄存器布局到IOB上; 1: 将双向Buffer连接的寄存器布局到IOB上。

分类

[Project](#)

描述

启用该选项，Place & Route 会将双向 Buffer 连接的寄存器布局到 IOB 上，默认为 1。

参数

<0|1>将双向 Buffer 连接的寄存器布局到 IOB 上使能控制。

示例

```
set_option -ioreg_in_iob 1
```

-replicate_resources

对高扇出的资源进行复制来降低扇出，获得较好的时序结果。

语法

-replicate_resources <0|1>

使用

名称	描述
<0 1>	0: 不对高扇出的资源进行复制来降低扇出; 1: 对高扇出的资源进行复制来降低扇出。

分类

[Project](#)

描述

启用该选项，Place & Route 会对高扇出的资源进行复制来降低扇出，获得较好的时序结果，默认为 0。

参数

<0|1>对高扇出的资源进行复制来降低扇出使能控制。

示例

```
set_option -replicate_resources 1
```

-serdesRetiming

在布局时将 logic 资源移近 SerDes node 用以平衡时序。

语法

-serdesRetiming <0|1>

使用

名称	描述
<0 1>	0: 在布局时不将logic资源移近SerDes node用以平衡时序; 1: 在布局时将logic资源移近SerDes node用以平衡时序。

分类

[Project](#)

描述

启用该选项，在布局时将 logic 资源移近 SerDes node 用以平衡时序，默认为 0。

参数

<0|1>在布局时将 logic 资源移近 SerDes node 用以平衡时序使能控制。

示例

```
set_option -serdesRetiming 1
```

-co-place_io_registers

共置输入输出寄存器使能控制。

语法

-co-place_io_registers <0|1>

使用

名称	描述
<0 1>	0: 不使用; 1: 使用。

分类

[Project](#)

描述

启用该选项，布局时采用共置输入输出寄存器的优化算法，默认为 0。

参数

<0|1>在布局时是否使用共置输入输出寄存器使能控制。

示例

```
set_option -co-place_io_registers 1
```

-clock_route_order

指定除时钟原语产生的时钟线之外的时钟线的绕线分配顺序。

语法

```
-clock_route_order <0|1>
```

使用

名称	描述
<0 1>	0: 根据net的扇出数量由多到少的顺序进行分配; 1: 根据频率由高到底的顺序进行分配。

分类

[Project](#)

描述

指定除时钟原语产生的时钟线之外的时钟线的绕线分配顺序，可选项有 0 和 1，默认值为 0。

参数

<0|1>指定除时钟原语产生的时钟线之外的时钟线的绕线分配顺序。

示例

```
set_option -clock_route_order 1
```

-route_maxfan

设置绕线最大扇出数目。

语法

```
-route_maxfan <value>
```

使用

名称	描述
< value>	设置绕线最大扇出数目。

分类

[Project](#)

描述

基于绕线优化，使用该选项设置绕线最大扇出数目，取值应为大于 0 且小于等于 100 的整数。器件为 GW1NZ-1/GW1N-2/GW1NR-2/GW1NZ-2/GW1N-1P5 时，选项默认值为 10，其他器件默认值为 23。

参数

<value>设置绕线最大扇出数目。

示例

```
set_option -route_maxfan 60
```

-correct_hold_violation

布线对时序 Hold 问题进行自动修复。

语法

```
-correct_hold_violation <0|1>
```

使用

名称	描述
<0 1>	0: 布线不对时序Hold问题进行自动修复; 1: 布线对时序Hold问题进行自动修复。

分类

[Project](#)

描述

启用该选项，布线会对时序 Hold 问题进行自动修复，默认为 1。

参数

<0|1>布线会对时序 Hold 问题进行自动修复使能控制。

示例

```
set_option -correct_hold_violation 1
```

-convert_sdp32_36_to_sdp16_18

将 32/36 位宽 SDPB/SDPX9B 转换为 2 个 16/18 位宽 SDPB/SDPX9B，仅 Arora V 器件支持该命令。

语法

```
-convert_sdp32_36_to_sdp16_18 <0|1>
```

使用

名称	描述
<0 1>	0: 不将32/36位宽SDPB/SDPX9B转换为2个16/18位宽SDPB/SDPX9B; 1: 将32/36位宽SDPB/SDPX9B转换为2个16/18位宽SDPB/SDPX9B。

分类

[Project](#)

描述

启用该选项会将 32/36 位宽 SDPB/SDPX9B 转换为 2 个 16/18 位宽 SDPB/SDPX9B，默认为 0。

参数

<0|1>将 32/36 位宽 SDPB/SDPX9B 转换为 2 个 16/18 位宽 SDPB/SDPX9B 使能控制。

示例

```
set_option -convert_sdp32_36_to_sdp16_18 1
```

-inc_place <0|auto|file>

增量布局。

语法

```
-inc_place <0|auto|file>
```

使用

名称	描述
<0 auto file >	0: 关闭增量布局; auto: 自动增量布局; file: 指定*.p文件进行增量布局。

分类

[Project](#)

描述

启用该选项，会使用增量布局，默认为 0。

参数

<0|auto|file >增量布局使能控制。

示例

```
set_option -inc_place auto
```

-inc_pnr <0|auto|file>

增量布局布线。

语法

```
-inc_pnr <0|auto|file>
```

使用

名称	描述
<0 auto file >	0: 关闭增量布局布线; auto: 自动增量布局布线; file: 指定*.p文件进行增量布局布线。

分类

[Project](#)

描述

启用该选项，会使用增量布局布线，默认为 0。

参数

<0|auto|file >增量布局布线使能控制。

示例

```
set_option -inc_pnr auto
```

注！

Place & Route 相关运行选项的详细用法，请参考 [SUG100, Gowin 云源软件用户指南](#) 中的 Place & Route 部分。

复用管脚属性配置

-use_jtag_as_gpio

将 JTAG 相关管脚复用为普通 IO 管脚。

语法

```
-use_jtag_as_gpio <0|1>
```

使用

名称	描述
<0 1>	0: 作为 JTAG 专用管脚; 1: 复用为普通 IO 管脚。

分类

[Project](#)

描述

将 JTAG 相关管脚复用为普通 IO 管脚，默认为 0。

参数

<0|1>将 JTAG 相关管脚复用为普通 IO 管脚使能控制。

示例

```
set_option -use_jtag_as_gpio 1
```

-use_sspi_as_gpio

将 SSPI 相关管脚复用为普通 IO 管脚。

语法

```
-use_ssapi_as_gpio <0|1>
```

使用

名称	描述
<0 1>	0: 作为SSPI专用管脚; 1: 复用为普通IO管脚。

分类

[Project](#)

描述

将 SSPI 相关管脚复用为普通 IO 管脚，默认为 0。

参数

<0|1>将 SSPI 相关管脚复用为普通 IO 管脚使能控制。

示例

```
set_option -use_sspi_as_gpio 1
```

-use_mspi_as_gpio

将 MSPI 相关管脚复用为普通 IO 管脚。

语法

-use_mspi_as_gpio <0|1>

使用

名称	描述
<0 1>	0: 作为MSPI专用管脚; 1: 复用为普通IO管脚。

分类

[Project](#)

描述

将 MSPI 相关管脚复用为普通 IO 管脚，默认为 0。

参数

<0|1>将 MSPI 相关管脚复用为普通 IO 管脚使能控制。

示例

```
set_option -use_mspi_as_gpio 1
```

-use_ready_as_gpio

将 READY 相关管脚复用为普通 IO 管脚。

语法

-use_ready_as_gpio <0|1>

使用

名称	描述
<0 1>	0: 作为READY专用管脚; 1: 复用为普通IO管脚。

分类

[Project](#)

描述

将 READY 相关管脚复用为普通 IO 管脚，默认为 0。

参数

<0|1>将 READY 相关管脚复用为普通 IO 管脚使能控制。

示例

```
set_option -use_ready_as_gpio 1
```

-use_done_as_gpio

将 DONE 相关管脚复用为普通 IO 管脚。

语法

```
-use_done_as_gpio <0|1>
```

使用

名称	描述
<0 1>	0: 作为 DONE 专用管脚; 1: 复用为普通 IO 管脚。

分类

[Project](#)

描述

将 DONE 相关管脚复用为普通 IO 管脚，默认为 0。

参数

<0|1>将 DONE 相关管脚复用为普通 IO 管脚使能控制。

示例

```
set_option -use_done_as_gpio 1
```

-use_reconfig_as_gpio

将 RECONFIG_N 相关管脚复用为普通 IO 管脚。

语法

```
-use_reconfig_as_gpio <0|1>
```

使用

名称	描述
<0 1>	0: 作为RECONFIG_N专用管脚; 1: 复用为普通IO管脚。

分类

[Project](#)

描述

将 RECONFIG_N 相关管脚复用为普通 IO 管脚，默认为 0。

参数

<0|1>将 RECONFIG_N 相关管脚复用为普通 IO 管脚使能控制。

示例

```
set_option -use_reconfign_as_gpio 1
```

-use_i2c_as_gpio

将 I2C 相关管脚复用为普通 IO 管脚。

语法

```
-use_i2c_as_gpio <0|1>
```

使用

名称	描述
<0 1>	0: 作为 I2C 专用管脚; 1: 复用为普通 IO 管脚。

分类

[Project](#)

描述

将 I2C 相关管脚复用为普通 IO 管脚，默认为 0。

参数

<0|1>将 I2C 相关管脚复用为普通 IO 管脚使能控制。

示例

```
set_option -use_i2c_as_gpio 1
```

BitStream 属性配置**-bit_format**

指定生成的码流文件内容的格式。

语法

```
-bit_format <txt|bin>
```

使用

名称	描述
<txt bin>	码流文件内容的格式。

分类

[Project](#)

描述

用于指定生成的码流文件内容的格式。默认为 bin。

参数

<txt|bin>生成的码流文件内容的格式。

示例

```
set_option -bit_format txt
```

-bit_crc_check

循环冗余校验。

语法

```
-bit_crc_check <0|1>
```

使用

名称	描述
<0 1>	0: 不启用循环冗余校验; 1: 启用循环冗余校验。

分类

[Project](#)

描述

对码流文件启用循环冗余校验。默认为 1。

参数

<0|1>对码流文件启用循环冗余校验。

示例

```
set_option -bit_crc_check 1
```

-bit_compress

对码流文件进行压缩。

语法

```
-bit_compress <0|1>
```

使用

名称	描述
<0 1>	0: 不对码流文件进行压缩; 1: 对码流文件进行压缩。

分类

[Project](#)

描述

对生成的码流文件进行压缩。默认为 1。

参数

<0|1>对码流文件进行压缩。

示例

```
set_option -bit_compress 1
```

-bit_encrypt

对码流文件进行加密处理。

语法

```
-bit_encrypt <0|1>
```

使用

名称	描述
<0 1>	0: 不进行加密处理; 1: 进行加密处理。

分类

[Project](#)

描述

对码流文件进行加密处理，仅支持晨熙家族器件。默认为 0。

参数

<0|1>对码流文件进行加密。

示例

```
set_option -bit_encrypt 1
```

-bit_encrypt_key

定义加密的密钥。

语法

```
-bit_encrypt_key <key>
```

使用

名称	描述
<key>	加密的密钥。

分类

[Project](#)

描述

该项与“-bit_encrypt”配合使用，用户可以对加密的密钥进行自定义。默认全为 0。

参数

<key>加密的密钥。

示例

<0|1>BSRAM 的初始值打印进码流文件中使能控制。

示例

```
set_option -bit_incl_bsram_init 1
```

-bsram_init_print_mode

将 BSRAM 的初始值打印进码流文件中的模式选择控制，仅 GW5A(S)(T)-138/GW5AT-75 器件支持该命令选项。

语法

```
-bsram_init_print_mode <default | partial>
```

使用

名称	描述
<default partial>	default: 将所有位置BSRAM的初始值都打印到码流文件中，没有被占用到的BSRAM位置的初始值会打印为0； partial: 将所占用位置BSRAM所在列的所有BSRAM的初始值都打印到码流文件中，该列中没有被占用到的BSRAM位置的初始值会打印为0。

分类

[Project](#)

描述

将 BSRAM 的初始值打印进码流文件中的模式选择控制，默认为 default，仅 GW5A(S)(T)-138/GW5AT-75 器件支持该命令选项。该项值为 default 时会将所有位置 BSRAM 的初始值都打印到码流文件中，没有被占用到的 BSRAM 位置的初始值会打印为 0。该项值为 partial 时会将所占用位置 BSRAM 所在的列的所有 BSRAM 的初始值都打印到码流文件中，该列中没有被占用到的 BSRAM 位置的初始值会打印为 0。选择 partial 时，需要先把 multiboot_mode 或 mspi_jump_mode 的 value 设为 quad。

参数

<default | partial>将 BSRAM 的初始值打印进码流文件中的模式选择控制。

示例

```
set_option -bsram_init_print_mode partial
```

-bg_programming

背景升级功能。

语法

```
-bg_programming <off | jtag | i2c | goconfig | userlogic | i2c_jtag_ssapi_qsspi | jtag_ssapi_qsspi>
```

使用

名称	描述
<off jtag i2c goconfig userlogic i2c_jtag_sspi_qsspi jtag_sspi_qsspi>	off: 不使用背景升级功能; jtag: 使用JTAG模式进行背景升级; i2c: 使用I2C模式进行背景升级; goconfig: 使用goConfig IP实现背景升级; userlogic: 使用FPGA内部逻辑进行背景升级; i2c_jtag_sspi_qsspi: 使用I2C/JTAG/SSPI/QSSPI模式进行背景升级; jtag_sspi_qsspi: 使用JTAG/SSPI/QSSPI模式进行背景升级。

分类

[Project](#)

描述

背景升级功能，在不中断 FPGA 芯片现有功能执行的前提下对 Flash 进行烧录。默认为 off。

参数

<off | jtag | i2c | goconfig | userlogic | i2c_jtag_sspi_qsspi | jtag_sspi_qsspi>背景升级方式。

示例

```
set_option -bg_programming userlogic
```

-hotboot

热启动模式。

语法

```
-hotboot <0|1>
```

使用

名称	描述
<0 1>	0: 不使用热启动模式; 1: 使用热启动模式。

分类

[Project](#)

描述

热启动模式使能控制。默认为 0。

参数

<0|1>热启动模式使能控制。

示例

```
set_option -hotboot 1
```

-i2c_slave_addr

设置 I2C 设备的地址。

语法

```
-i2c_slave_addr <value>
```

使用

名称	描述
<value>	I2C 设备的地址。

分类

[Project](#)

描述

设置 I2C 设备的地址，可配置值范围为 00~7F。默认为 00。

参数

<value>I2C 设备的地址。

示例

```
set_option -i2c_slave_addr 2F
```

-secure_mode

启用安全模式。

语法

```
-secure_mode <0|1>
```

使用

名称	描述
<0 1>	0: 不启用安全模式; 1: 启用安全模式。

分类

[Project](#)

描述

启用安全模式，此时 JTAG 管脚为 GPIO，码流文件只能对设备编程一次。默认为 0。

参数

<0|1>启用安全模式使能控制。

示例

```
set_option -secure_mode 1
```

-loading_rate

AutoBoot 配置模式和 MSPI 配置模式下，码流数据从 Flash 到 SRAM 的加载速度。

语法

```
-loading_rate <value>
```

使用

名称	描述
<value>	AutoBoot配置模式和MSPI配置模式下，码流数据从Flash到SRAM的加载速度。

分类

[Project](#)

描述

AutoBoot 配置模式和 MSPI 配置模式下，码流数据从 Flash 到 SRAM 的加载速度，默认为 2.500MHz。

参数

<value>码流数据从 Flash 到 SRAM 的加载速度。

示例

```
set_option -loading_rate 21.000MHz
```

-seu_handler

启用单粒子翻转处理器。

语法

```
-seu_handler <0|1>
```

使用

名称	描述
<0 1>	0: 不启用单粒子翻转处理器; 1: 启用单粒子翻转处理器。

分类

[Project](#)

描述

启用单粒子翻转处理器。默认为 0。

参数

<0|1>启用单粒子翻转处理器使能控制。

示例

```
set_option -seu_handler 1
```

-seu_handler_mode

选择 SEU Handler 启动或停止的模式。

语法

```
-seu_handler_mode <auto|userlogic>
```

使用

名称	描述
<auto userlogic>	auto: 芯片唤醒后自动启用单粒子翻转处理器; userlogic: 使用逻辑启用或停止单粒子翻转处理器。

分类

[Project](#)

描述

选择 SEU Handler 启动或停止的模式，默认为 auto。

参数

< auto|userlogic>SEU Handler 启动或停止的模式。

示例

```
set_option -seu_handler_mode userlogic
```

-seu_handler_checksum

启用单粒子翻转处理、检验、计算和对比。

语法

```
-seu_handler_checksum <0|1>
```

使用

名称	描述
<0 1>	0: 不启用单粒子翻转处理、检验、计算和对比; 1: 启用单粒子翻转处理、检验、计算和对比。

分类

[Project](#)

描述

启用单粒子翻转处理、检验、计算和对比。默认为 0。

参数

<0|1>启用单粒子翻转处理、检验、计算和对比使能控制。

示例

```
set_option -seu_handler_checksum 1
```

-error_detection

仅启用错误检验。

语法

-error_detection <0|1>

使用

名称	描述
<0 1>	0: 不启用错误检验; 1: 仅启用错误检验。

分类

[Project](#)

描述

仅启用错误检验。默认为 0。

参数

<0|1>仅启用错误检验使能控制。

示例

```
set_option -error_detection 1
```

-error_detection_correction

启用错误检验和更正。

语法

-error_detection_correction <0|1>

使用

名称	描述
<0 1>	0: 不启用错误检验和更正; 1: 启用错误检验和更正。

分类

[Project](#)

描述

启用错误检验和更正。默认为 0。

参数

<0|1>启用错误检验和更正使能控制。

示例

```
set_option -error_detection_correction 1
```

-stop_seu_handler

停止 SEU Handler。

语法

`-stop_seu_handler <0|1>`

使用

名称	描述
<0 1>	0: 当检测到不可纠正的ECC错误或CRC校验和不匹配错误时, 不停止SEU Handler; 1: 当检测到不可纠正的ECC错误或CRC校验和不匹配错误时, 停止SEU Handler。

分类

[Project](#)

描述

当检测到不可纠正的 ECC 错误或 CRC 校验和不匹配错误时, 停止 SEU Handler。默认为 0。

参数

<0|1>停止 SEU Handler 使能控制。

示例

```
set_option -stop_seu_handler 1
```

-osc_div

设置扩展控制寄存器的分频比。

语法

```
-osc_div <4|8|16|32>
```

使用

名称	描述
<4 8 16 32>	4: 扩展控制寄存器的分频比设置为4; 8: 扩展控制寄存器的分频比设置为8; 16: 扩展控制寄存器的分频比设置为16; 32: 扩展控制寄存器的分频比设置为32。

分类

[Project](#)

描述

设置扩展控制寄存器的分频比。默认为 8。

参数

<4|8|16|32>扩展控制寄存器的分频比。

示例

```
set_option -osc_div 8
```

-error_injection

启用错误注入。

语法

`-error_injection <0|1>`

使用

名称	描述
<code><0 1></code>	0: 不启用错误注入; 1: 启用错误注入。

分类

[Project](#)

描述

启用错误注入。默认为 0。

参数

`<0|1>` 启用错误注入使能控制。

示例

```
set_option -error_injection 1
```

-ext_cclk

启用外部主配置时钟。

语法

`-ext_cclk <0|1>`

使用

名称	描述
<code><0 1></code>	0: 不启用外部主配置时钟; 1: 启用外部主配置时钟。

分类

[Project](#)

描述

启用外部主配置时钟。默认为 0。

参数

`<0|1>` 启用外部主配置时钟使能控制。

示例

```
set_option -ext_cclk 1
```

-ext_cclk_div

设置分频器参数。

语法

-ext_cclk_div <value>

使用

名称	描述
<value>	分频器参数。

分类

[Project](#)

描述

设置分频器参数。

参数

< value>设置分频器参数。

示例

```
set_option -ext_cclk_div 4
```

-multi_boot

Multi Boot 使能控制。

语法

-multi_boot <0|1>

使用

名称	描述
<0 1>	0: 不启用Multi Boot; 1: 启用Multi Boot。

分类

[Project](#)

描述

Multi Boot 使能控制，默认为 0。

参数

<0|1> Multi Boot 使能控制。

示例

```
set_option -multi_boot 1
```

-multiboot_address_width

配置 SPI Flash 地址位宽。

语法

-multiboot_address_width<24|32>

使用

名称	描述
<24 32>	24: SPI Flash地址位宽设为24; 32: SPI Flash地址位宽设为32。

分类

[Project](#)

描述

配置 SPI Flash 地址位宽，默认为 24。

参数

<24|32> SPI Flash 地址位宽。

示例

```
set_option -multiboot_address_width 32
```

-multiboot_spi_flash_address

指定 SPI Flash 地址。

语法

```
-multiboot_spi_flash_address <value>
```

使用

名称	描述
<value>	SPI Flash 地址。

分类

[Project](#)

描述

指定 SPI Flash 地址。SPI Flash 地址是指下一次 multiboot 时，加载码流文件的起始地址，默认为 000000。

参数

< value>SPI Flash 地址。

示例

```
set_option -multiboot_spi_flash_address 000110
```

-multiboot_mode

配置 SPI Flash 地址访问模式。

语法

```
-multiboot_mode <single | fast | dual | quad>
```

使用

名称	描述
< single fast dual quad >	single: 使用single模式; fast: 使用fast模式; dual: 使用dual模式; quad: 使用quad模式。

分类

[Project](#)

描述

配置 SPI Flash 地址访问模式，默认为 single。

参数

< single | fast | dual | quad > SPI Flash 地址访问模式。

示例

```
set_option -multiboot_mode single
```

-mspi_jump

MSPI JUMP 使能控制。

语法

-mspi_jump<0|1>

使用

名称	描述
<0 1>	0: 不启用MSPI JUMP; 1: 启用MSPI JUMP。

分类

[Project](#)

描述

MSPI JUMP 使能控制，默认为 0。

参数

<0|1> MSPI JUMP 使能控制。

示例

```
set_option -mspi_jump 1
```

-merge_jumpbit

将 MSPI JUMP 码流文件合并到通用的码流文件中。

语法

-merge_jumpbit <0|1>

使用

名称	描述
<0 1>	0: 不合并码流文件; 1: 合并码流文件。

分类

[Project](#)

描述

将 MSPI JUMP 码流文件合并到通用的码流文件中，默认为 0。

参数

<0|1> MSPI JUMP 码流文件合并到通用的码流文件中使能控制。

示例

```
set_option -merge_jumpbit 1
```

-mspijump_address_width

配置 SPI Flash 地址位宽。

语法

```
-mspijump_address_width <24|32>
```

使用

名称	描述
<24 32>	24: SPI Flash地址位宽设为24; 32: SPI Flash地址位宽设为32。

分类

[Project](#)

描述

配置 SPI Flash 地址位宽，默认为 24。

参数

<24|32> SPI Flash 地址位宽。

示例

```
set_option - mspijump_address_width 32
```

-mspijump_spi_flash_address

指定 SPI Flash 地址。

语法

```
-mspijump_spi_flash_address <value>
```

使用

名称	描述
<value>	SPI Flash地址。

分类

[Project](#)

描述

指定 SPI Flash 地址，默认为 000000。

参数

<value>SPI Flash 地址。

示例

```
set_option - mspijump_spi_flash_address 000110
```

-mspijump_mode

配置 SPI Flash 地址访问模式。

语法

-mspijump_mode <single | fast | dual | quad>

使用

名称	描述
< single fast dual quad >	single: 使用single模式; fast: 使用fast模式; dual: 使用dual模式; quad: 使用quad模式。

分类

[Project](#)

描述

配置 SPI Flash 地址访问模式，默认为 single。

参数

< single | fast | dual | quad > SPI Flash 地址访问模式。

示例

```
set_option -mspijump_mode single
```

-program_done_bypass

转发新的码流数据。

语法

-program_done_bypass <0|1>

使用

名称	描述
<0 1>	0: 不启用该功能; 1: 启用该功能。

分类

[Project](#)

描述

配置该选项后，在 **Done Final** 内部信号生效时，同时外部 **Done Pin** 保持低电平，使码流加载完成后可以转发新的码流数据，默认为 0。

参数

<0|1> 转发新的码流数据使能控制。

示例

```
set_option -program_done_bypass 1
```

-power_on_reset_monitor

上电复位。

语法

```
-power_on_reset_monitor <0|1>
```

使用

名称	描述
<0 1>	0: 不启用上电复位功能; 1: 启用上电复位功能。

分类

[Project](#)

描述

上电复位功能的使能控制，默认为 0。

参数

<0|1> 上电复位功能的使能控制。

示例

```
set_option -power_on_reset_monitor 1
```

-turn_off_bg

Bandgap 功能。

语法

```
-turn_off_bg <0|1>
```

使用

名称	描述
<0 1>	0: 启用Bandgap功能; 1: 不启用Bandgap功能。

分类

[Project](#)

描述

Bandgap 功能的使能控制。默认为 0。

参数

<0|1> Bandgap 功能的使能控制。

示例

```
set_option -turn_off_bg 1
```

-wakeup_mode

Wake Up Mode 使能控制。

语法

-wakeup_mode <0|1>

使用

名称	描述
<0 1>	0: 关闭Wake Up Mode的使能控制; 1: 打开Wake Up Mode的使能控制。

分类

[Project](#)

描述

Wake Up Mode 的使能控制。默认为 0。

参数

<0|1> Wake Up Mode 的使能控制。

示例

```
set_option -wakeup_mode 1
```

-user_code

自定义 User Code。

语法

-user_code <default|value>

使用

名称	描述
<default value>	定义 User Code 值。

分类

[Project](#)

描述

用户可以自定义 User Code。默认为 default(00000000)。

参数

<default|value>自定义 User Code。

示例

```
set_option -user_code 00000010
```

注！

BitStream 相关选项的详细解释，请参考 [SUG100, Gowin 云源软件用户指南](#) 中的 BitStream 部分。

Unused Pin 属性配置

-unused_pin

对未使用的 GPIO 设置不同的 IO 属性。

语法

-unused_pin <default|open_drain>

使用

名称	描述
<default open_drain>	default: 所有未使用的GPIO会配置为输入三态弱上拉； open_drain: 所有未使用的GPIO会配置为输出类型，并且OPEN DRAIN配置为ON。

分类

[Project](#)

描述

对未使用的 GPIO 设置不同的 IO 属性。

参数

<default|open_drain>对未使用的 GPIO 设置不同的 IO 属性。

示例

```
set_option -unused_pin open_drain
```

注！

Unused Pin 相关选项的详细解释，请参考 [SUG100, Gowin 云源软件用户指南](#)中的 Unused Pin 部分。

3.3.21 set_property

配置 IP 的选项。

语法

set_property [-dict <args>] <name> <value> <objects>

使用

名称	描述
[-dict]	要设置的（name/value）对的选项列表
<name>	要设置的选项的名称，使用-dict时无效
<value>	要设置的选项的值，使用-dict时无效
<objects>	要设置选项的IP

分类

[IPFlow](#)

描述

配置指定 IP 的选项及其对应的选项值。

参数

- [-dict]指定一个包含多对选项及对应的选项值的字典，通过（<name> <value>）指定一对，多对之间通过空格隔开，字典需要通过大括号{}括起来。
- <name>指定要配置的选项的名称，描述形式为 CONFIG.property，property 指代的是选项名称；
- <value>指定要配置的选项对应的选项值，需要根据属性类型判断该值，以确保该值是有效的，如果选项值是字符串形式，统一规范为原形式；
- <objects>指定要配置选项的一个或多个 IP，一个 IP 通过[get_ips module_name]指定，如果是指定多个 IP，则使用[get_ips module_name0 module_name1 ...]。

示例

使用-dict 选项在当前设计上一次指定多个 IP 配置选项：

```
set_property -dict {CONFIG.Data_Width 16 CONFIG.Write_Depth 1024 CONFIG.Read_Depth 1024} [get_ips FIFO_Top]
```

使用（name, value, objects）在当前设计上指定一个 IP 配置选项：

```
set_property CONFIG.Data_Width {16} [get_ips FIFO_Top]
```

该示例展示了如何设置一个包含波折号“-”或空格的选项值：

```
set_property {CONFIG.Almost_Full_Type} {Full-Single Threshold
```

Constant Parameter} [get_ips FIFO_Top]

注！

某些情况下选项值中可能包含特殊字符，比如破折号“-”或空格，这有可能导致该选项值被错误解析，这种情况下必须使用大括号{}将选项值括起来。

另请参阅

- [list_property](#)
- [report_property](#)

3.3.22 source

在云源的 tcl 命令编辑窗口中或启动命令行模式后，使用该命令可以执行 tcl 脚本。文件路径形式请参考 [add_file](#)。

语法

```
source <file>
```

使用

名称	描述
<file>	要执行的tcl脚本。

分类

[IPFlow](#), [Project](#)

描述

对未使用的 GPIO 设置不同的 IO 属性。

参数

<file> 要被执行的 Tcl 脚本文件。

示例

```
source project.tcl
source D:/gowin_project/project.tcl
source D:\\gowin_project\\project.tcl
```

3.3.23 write_ip_tcl

导出 Tcl 脚本，该脚本可以重新生成给定的 IP。

语法

```
write_ip_tcl [-ip_name <newModuleName>] [-multiple_files] [-force]
[<tcl_filename>] <objects>
```

使用

名称	描述
<code>[-ip_name]</code>	设置 IP module 名称
<code>[-multiple_files]</code>	为每个 IP 创建一个.tcl 文件
<code>[-force]</code>	覆盖已有文件
<code>[<tcl_filename>]</code>	导出的 tcl 文件
<code><objects></code>	指定需要导出 tcl 文件的 IP

分类

[IPFlow](#)

描述

该命令为指定的 IP 导出 Tcl 脚本文件，便于通过 Tcl 脚本再次生成 IP，文件路径形式请参考 [add_file](#)。

参数

- `[<tcl_filename>]` 生成的 Tcl 脚本文件名称，不指定时使用当前工程名称命名；
- `<objects>`指定要生成 Tcl 脚本的一个或多个 IP，一个 IP 通过`[get_ips module_name]`指定，如果是指定多个 IP，则使用`[get_ips module_name0 module_name1 ...]`。
- `[-ip_name <newModuleName>]`重命名生成的 Tcl 脚本中的 IP 的 `module_name`，只能指定一个对象；
- `[-multiple_files]`为指定的所有 IP 生成 Tcl 脚本文件，脚本名称以对应 IP 的 `module_name` 命名，该选项与`-ip_name` 以及`<tcl_filename>`互斥；
- `[-force]`覆盖已有的同名的 Tcl 文件。

示例

该示例为 IP FIFO_Top 导出 Tcl 文件，在运行 `source` 命令时创建的 IP module 名称为 FIFO_Top_new:

```
write_ip_tcl -ip_name FIFO_Top_new [get_ips FIFO_Top]
```

该示例为工程中指定的每个 IP 编写独立的 Tcl 文件:

```
write_ip_tcl -multiple_files [get_ips FIFO_Top FIFO_Top_1]
```

该示例将工程中的指定的多个 IP 写入一个 Tcl 文件:

```
write_ip_tcl [get_ips FIFO_Top FIFO_Top_1] my_fifo.tcl
```

4 Tcl 生成 IP 流程

Tcl 生成 IP，需要在工程下执行，因此首先通过 `create_project` 命令创建工程或 `open_project` 命令打开已有工程，后续流程如下：

1. 通过 `create_ipc` 命令创建指定 IP 的默认配置的 IPC 文件或通过 `read_ipc` 命令打开已有的 IPC 文件；
2. 可通过 `set_property` 命令修改 IP 配置选项，IP 有哪些配置选项可通过 `list_property` 或 `report_property` 命令查看，配置选项的选项值可通过对应 IP 的用户指南文档查看；
3. 配置完成后，通过 `generate_target` 命令生成 IP 设计文件并将其添加到工程中，用户还可以通过 `write_ip_tcl` 命令导出 Tcl 脚本文件，便于通过 Tcl 脚本再次生成 IP。

示例

以 ROM IP 为例介绍 Tcl 生成 IP 流程：

```
set projectName "rom_tcl"
set dirPath "E:/tcl"
set partNumber "GW1N-LV9LQ144C6/I5"
set deviceVersion "C"
set initFile "E:/init_file/rom.mi"

#create project

create_project -name $projectName -dir $dirPath -pn $partNumber -
device_version $deviceVersion

#####Generate ROM IP#####

#create ROM IP

create_ipc -name rom -module_name Gowin_ROM -language Verilog
-file_name gowin_rom

#set the option value of specified options

set_property CONFIG.Data_Width 8 [get_ips Gowin_ROM]
set_property CONFIG.Memory_Initialization_File $initFile [get_ips
```

Gowin_ROM]

#generate IP file and add to project

`generate_target` [`get_ips` Gowin_ROM]

